iMovies – Movie Ticket Booking System

Full Stack Development with MERN
Project Documentation

Student: Nidhi Rathore (Individual Project)
Institute: Medi-Caps University
Department: Computer Science and Engineering (CSE)
Course: Full Stack Development – MERN Stack
Submission Date: 22 October 2026

---

1. Introduction

Project Title: iMovies – Movie Ticket Booking System

iMovies is a full-stack web application for finding movies, checking showtimes, selecting seats and managing movie ticket bookings. The React frontend talks to a Node.js/Express backend, and the data is stored in MongoDB. The project includes login, seat booking, booking management and role-based admin access.

Student: Nidhi Rathore (Individual Project)
Institute: Medi-Caps University
Department: Computer Science and Engineering (CSE)

2. Overview of the Project

Purpose

iMovies gives users one place to browse movies, see showtimes, pick seats, complete a demo checkout and manage their bookings.

Main Features

- User registration, login and logout.
- Browse currently showing and upcoming movies, search and filter them, and open movie details.
- View theatres and showtimes, select seats and place a temporary hold on them.
- Simulated payment (success or failure), booking details and booking history.
- Cancellation of eligible bookings with a demo refund.
- Admin functions for movies, theatres, screens, shows, bookings and registered users.

3. Architecture

- **Frontend:** React 18 with Vite, JavaScript, Bootstrap 5, React Router and Axios.
- **Backend:** Node.js 18+ and an Express.js REST API.
- **Database:** MongoDB, accessed through Mongoose.
- **Authentication and security:** JWT stored in an HTTP-only cookie, password hashing, request validation, Helmet, rate limiting and admin role checks.

4. Setup Instructions

Prerequisites

- Node.js 18 or later and npm.
- Git.
- A MongoDB database that you can reach, such as MongoDB Atlas.

Installation

Clone the repository and install the packages in the client and server folders separately.

```
git clone https://github.com/rnidhi4/iMovies.git
cd iMovies

cd client
npm install

cd ../server
npm install
```

Environment variables

In the server folder, create a .env file from .env.example and fill in the values. The required ones are MONGODB_URI, JWT_SECRET and CLIENT_URL. The frontend can optionally use VITE_API_URL. Keep real credentials private and never commit the .env file.

5. Folder Structure

The repository has a React client, an Express server and documentation folders.

```
iMovies/
  client/src/    - api, components, context, hooks, layouts, pages, routes, utils
  server/src/    - config, controllers, middleware, models, routes, services, utils
  server/scripts/ - backend utility scripts
  docs/ and Phase wise Doc/ - project documentation and testing material
  README.md      - setup and project overview
```

6. Running the Application

Run the backend and the frontend in two separate terminals.

Backend terminal

```
cd server
npm run dev
```

Frontend terminal

```
cd client
npm run dev
```

By default the backend runs at http://localhost:5000 and the Vite frontend at http://localhost:5173. The health check is at /api/v1/health. The ports can be different if you change the configuration.

7. API Documentation

All endpoints use the /api/v1 prefix. For example, the health check is GET /api/v1/health.

Area	Method and endpoint	Purpose
Health	GET /health	Check that the backend is running.
Authentication	POST /auth/register POST /auth/login POST /auth/logout GET /auth/me	Register, log in and out, and get the current user.
Movies	GET /movies GET /movies/:id GET /movies/:id/shows	List movies, view one movie and get its shows.
Theatres and shows	GET /theatres GET /shows/:id GET /shows/:id/seats	Theatres, show details and seat availability.
Bookings	GET /bookings POST /bookings/hold GET /bookings/:id POST /bookings/:id/cancel	List bookings, hold seats, view a booking and cancel it when eligible.
Demo payment	POST /payments/demo/:bookingId	Simulated payment. No real payment is processed.
Admin	GET /admin/stats GET /admin/users GET /admin/bookings POST /admin/bookings/:id/cancel	Dashboard statistics, users, bookings and admin cancellation.
Admin management	POST, PUT, DELETE /admin/movies GET, POST, PUT, DELETE /admin/theatres POST, DELETE /admin/screens GET, POST, PUT /admin/shows	Manage movies, theatres, screens and shows. Shows cannot be deleted from the admin interface.

8. Authentication

Users register and log in through the authentication API. Passwords are stored as hashes, and the JWT is kept in an HTTP-only cookie. Protected routes need a logged-in user, admin operations need the admin role, and logout clears the cookie.

Payment in this project is simulated. No real payment is processed and no card details are collected.

9. User Interface

The interface uses a dark theme with navigation links, a search box, language and genre filters, rating badges and movie cards. Screenshots of the application are shown below.

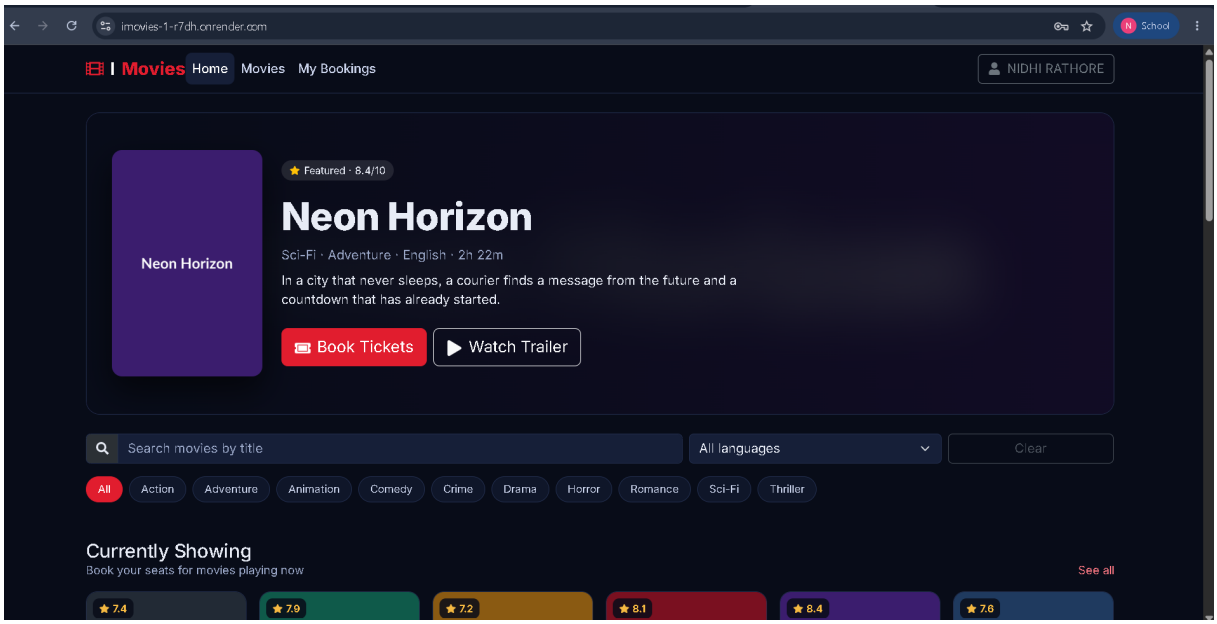


Figure 1. Home page with the featured movie, search, language filter and genre buttons.

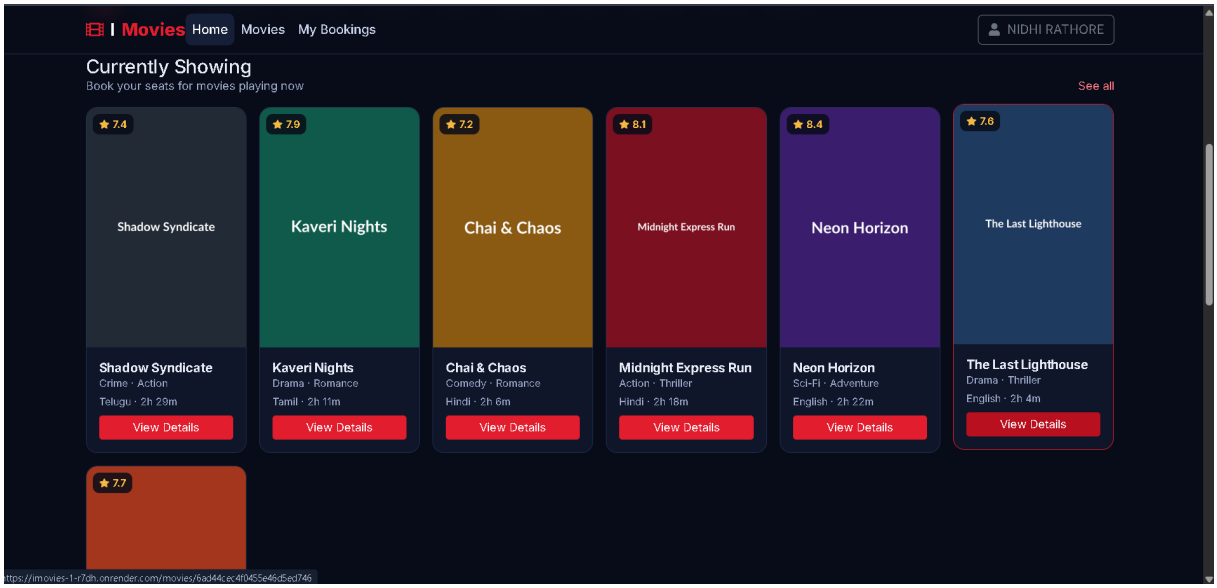


Figure 2. Currently showing movies with ratings, genres, languages and details buttons.

10. Testing

Functional test cases are listed in Phase wise Doc/Performance Testing/02-Functional-Test-Cases.md. They cover registration, login and logout, movie browsing, showtimes, seat holds, demo payment success and failure, booking history, cancellation and admin authorization.

No test results are recorded in this document yet. Automated testing and performance measurement are still pending.

11. Screenshots and Demo

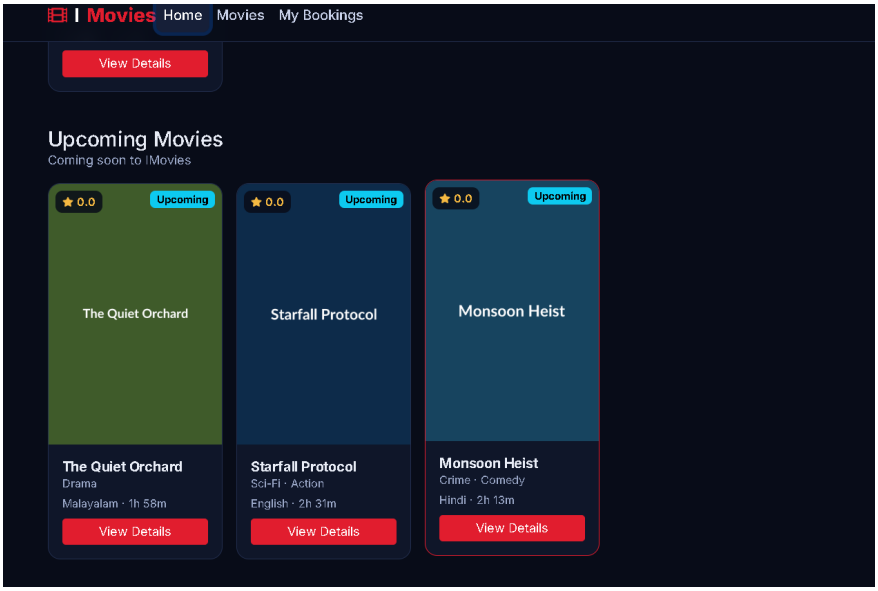


Figure 3. Upcoming movies section with status labels and details buttons.

GitHub repository: https://github.com/rnidhi4/iMovies

Live demo (Render): https://imovies-1-r7dh.onrender.com

12. Known Issues

- Payments and refunds are simulated. There is no real payment gateway.
- Email or SMS notifications and QR-code tickets are not included.
- Shows cannot be deleted from the admin interface.
- Customer show listings only cover the upcoming seven days.
- Performance measurements have not been recorded yet.

13. Future Enhancements

- Connect a real payment provider, starting in sandbox mode.
- Add email confirmations, QR-code tickets and downloadable invoices.
- Add movie reviews and a watchlist.
- Add more functional, automated and concurrent seat-booking tests.

- Measure and improve API response times and page performance.
- Improve accessibility and mobile responsiveness.